

CINESTREAM BACKEND DATABASE PROJECT

Submitted By

Name: Gaurav Kumar

Roll No: ADT23SOCB0393

Course: Database Management Systems

Date: 11-06-2026

1. Introduction

CineStream is an online movie streaming platform that offers users access to a large collection of movies across different genres. As the platform grows, maintaining user information, movie catalogs, and watch history using spreadsheets becomes inefficient and error-prone.

To solve this problem, a relational database management system (RDBMS) is implemented. The database centralizes all information, reduces redundancy, ensures data integrity, and enables efficient reporting and analysis.

2. Problem Statement

The CineStream management team currently stores movie details, user information, and viewing records in disconnected spreadsheets.

This approach creates several challenges:

- Data duplication and inconsistency
- Difficulty in tracking user activities
- Slow report generation
- Poor scalability as the user base grows
- Lack of meaningful business insights

Therefore, a centralized database system is required to manage daily operations and support analytical reporting.

3. Objectives

The main objectives of this project are:

1. Design a relational database for CineStream.
2. Store movie catalog information efficiently.
3. Manage registered user accounts.
4. Track movie viewing activities.

5. Perform CRUD (Create, Read, Update, Delete) operations.
 6. Generate analytical reports using SQL JOIN operations.
 7. Improve data consistency and business decision-making.
-

4. Database Design

The database consists of three tables:

1. Movies

Stores information about movies available on the platform.

2. Users

Stores registered user details.

3. WatchLog

Stores viewing history and acts as a junction table between Users and Movies.

5. Entity Relationship Overview

Users and Movies have a Many-to-Many relationship.

A user can watch multiple movies.

A movie can be watched by multiple users.

This relationship is managed through the WatchLog table.

6. Table Creation

6.1 Movies Table

SQL Query

```
CREATE TABLE Movies (  
    movie_id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(255) NOT NULL,  
    genre VARCHAR(100) NOT NULL,  
    release_year INT,  
    rental_rate DECIMAL(5,2)  
);
```

Output Screenshot

```
mysql> USE library;
Database changed
mysql> CREATE TABLE Movies (
    -> movie_id INT AUTO_INCREMENT PRIMARY KEY,
    -> title VARCHAR(255) NOT NULL,
    -> genre VARCHAR(100) NOT NULL,
    -> release_year INT,
    -> rental_rate DECIMAL (5,2)
    -> );
Query OK, 0 rows affected (0.10 sec)
```

6.2 Users Table

SQL Query

```
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    join_date DATE,
    status VARCHAR(20)
);
```

```
mysql> CREATE TABLE Users (
    -> user_id INT AUTO_INCREMENT PRIMARY KEY,
    -> username VARCHAR(100) UNIQUE NOT NULL,
    -> email VARCHAR(255) UNIQUE NOT NULL,
    -> join_date DATE,
    -> status VARCHAR (20)
    -> );
Query OK, 0 rows affected (0.10 sec)
```

6.3 WatchLog Table

SQL Query

```
CREATE TABLE WatchLog (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    movie_id INT,
    watch_date DATETIME,
```

```

duration_minutes INT,

FOREIGN KEY (user_id) REFERENCES Users(user_id),

FOREIGN KEY (movie_id) REFERENCES Movies(movie_id)

);

```

```

mysql> CREATE TABLE Watchlog (
  -> log_id INT AUTO_INCREMENT PRIMARY KEY,
  -> user_id INT,
  -> movie_id INT,
  -> watch_date DATETIME,
  -> duration_minutes INT,
  -> FOREIGN KEY(user_id) REFERENCES Users(user_id),
  -> FOREIGN KEY(movie_id) REFERENCES Movies(movie_id)
  -> );
Query OK, 0 rows affected (0.08 sec)

```

7. Sample Data Insertion

7.1 Insert Data into Movies Table

-- Paste your INSERT query here

Output Screenshot

```

mysql> INSERT INTO Movies (title, genre, release_year, rental_rate) VALUES
  -> ('Tarzan', 'Sci-Fi', 2014, 4.99),
  -> ('Nirahua Hindustani', 'Action', 2010, 5.99),
  -> ('Rickshawala', 'Action', 2008, 4.49),
  -> ('Titanic', 'Romance', 1997, 2.99),
  -> ('Titanic', 'Romance', 1997, 2.99),
  -> ('The Notebook', 'Romance', 2004, 2.49),
  -> ('Joker', 'Drama', 2019, 3.99),
  -> ('Forrest Gump', 'Drama', 1994, 2.99),
  -> ('The Hangover', 'Comedy', 2009, 2.99),
  -> ('Deadpool', 'Comedy', 2016, 3.49);
Query OK, 10 rows affected (0.03 sec)
Records: 10 Duplicates: 0 Warnings: 0

```

```

mysql> SELECT * FROM Movies;
+-----+-----+-----+-----+-----+
| movie_id | title           | genre  | release_year | rental_rate |
+-----+-----+-----+-----+-----+
| 1        | Tarzan          | Sci-Fi | 2014         | 4.99        |
| 2        | Nirahua Hindustani | Action | 2010         | 5.99        |
| 3        | Rickshawala     | Action | 2008         | 4.49        |
| 4        | Titanic         | Romance | 1997         | 2.99        |
| 5        | Titanic         | Romance | 1997         | 2.99        |
| 6        | The Notebook    | Romance | 2004         | 2.49        |
| 7        | Joker           | Drama  | 2019         | 3.99        |
| 8        | Forrest Gump    | Drama  | 1994         | 2.99        |
| 9        | The Hangover    | Comedy | 2009         | 2.99        |
| 10       | Deadpool        | Comedy | 2016         | 3.49        |
+-----+-----+-----+-----+-----+
10 rows in set (0.00 sec)

```

7.2 Insert Data into Users Table

-- Paste your INSERT query here

Output Screenshot

```
mysql> INSERT INTO Users (username, email, join_date, status) VALUES
-> ('johndoe123', 'john@example.com', '2025-02-10', 'Inactive'),
-> ('alice01', 'alice@example.com', '2025-03-15', 'Active'),
-> ('bob22', 'bob@example.com', '2024-11-20', 'Active'),
-> ('charlie77', 'charlie@example.com', '2025-01-18', 'Active'),
-> ('david89', 'david@example.com', '2024-09-05', 'Inactive'),
-> ('emma55', 'emma@example.com', '2025-04-01', 'Active'),
-> ('frank90', 'frank@example.com', '2025-05-12', 'Active'),
-> ('grace11', 'grace@example.com', '2023-12-25', 'Inactive'),
-> ('harry33', 'harry@example.com', '2025-02-28', 'Active'),
-> ('isabella44', 'isabella@example.com', '2025-06-01', 'Active');
Query OK, 10 rows affected (0.02 sec)
Records: 10 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Users;
+-----+-----+-----+-----+-----+
| user_id | username | email | join_date | status |
+-----+-----+-----+-----+-----+
| 1 | johndoe123 | john@example.com | 2025-02-10 | Inactive |
| 2 | alice01 | alice@example.com | 2025-03-15 | Active |
| 3 | bob22 | bob@example.com | 2024-11-20 | Active |
| 4 | charlie77 | charlie@example.com | 2025-01-18 | Active |
| 5 | david89 | david@example.com | 2024-09-05 | Inactive |
| 6 | emma55 | emma@example.com | 2025-04-01 | Active |
| 7 | frank90 | frank@example.com | 2025-05-12 | Active |
| 8 | grace11 | grace@example.com | 2023-12-25 | Inactive |
| 9 | harry33 | harry@example.com | 2025-02-28 | Active |
| 10 | isabella44 | isabella@example.com | 2025-06-01 | Active |
+-----+-----+-----+-----+-----+
10 rows in set (0.01 sec)
```

7.3 Insert Data into WatchLog Table

-- Paste your INSERT query here

Output Screenshot

```
mysql> INSERT INTO WatchLog (user_id, movie_id, watch_date, duration_minutes) VALUES
-> (1, 1, '2026-01-01 19:00:00', 169),
-> (2, 2, '2026-01-02 20:00:00', 148),
-> (3, 3, '2026-01-03 18:30:00', 152),
-> (4, 4, '2026-01-04 21:15:00', 181),
-> (5, 5, '2026-01-05 17:45:00', 120),
-> (6, 6, '2026-01-06 19:30:00', 110),
-> (7, 7, '2026-01-07 20:45:00', 122),
-> (8, 8, '2026-01-08 16:00:00', 142),
-> (9, 9, '2026-01-09 22:00:00', 100),
-> (10, 10, '2026-01-10 18:00:00', 108);
Query OK, 10 rows affected (0.01 sec)
Records: 10 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM WatchLog;
```

log_id	user_id	movie_id	watch_date	duration_minutes
1	1	1	2026-01-01 19:00:00	169
2	2	2	2026-01-02 20:00:00	148
3	3	3	2026-01-03 18:30:00	152
4	4	4	2026-01-04 21:15:00	181
5	5	5	2026-01-05 17:45:00	120
6	6	6	2026-01-06 19:30:00	110
7	7	7	2026-01-07 20:45:00	122
8	8	8	2026-01-08 16:00:00	142
9	9	9	2026-01-09 22:00:00	100
10	10	10	2026-01-10 18:00:00	108

```
10 rows in set (0.00 sec)
```

8. CRUD Operations

8.1 CREATE Operation

Requirement

Add a new movie "Interstellar" to the Movies table.

Query

```
INSERT INTO Movies (title, genre, release_year, rental_rate)
```

```
VALUES ('Interstellar', 'Sci-Fi', 2014, 4.99);
```

```
mysql> SELECT * FROM Movies;
```

movie_id	title	genre	release_year	rental_rate
1	Tarzan	Sci-Fi	2014	4.99
2	Nirahua Hindustani	Action	2010	5.99
3	Rickshawala	Action	2008	4.49
4	Titanic	Romance	1997	2.99
5	Titanic	Romance	1997	2.99
6	The Notebook	Romance	2004	2.49
7	Joker	Drama	2019	3.99
8	Forrest Gump	Drama	1994	2.99
9	The Hangover	Comedy	2009	2.99
10	Deadpool	Comedy	2016	3.49

```
10 rows in set (0.00 sec)
```

8.2 READ Operation

Requirement

Display all active users who joined after January 1, 2025.

Query

```
SELECT *
```

```
FROM Users
```

WHERE join_date > '2025-01-01'

AND status = 'Active';

Output Screenshot

```
mysql> SELECT * FROM Users WHERE join_date > '2025-01-01' AND status = 'Active';
```

user_id	username	email	join_date	status
2	alice01	alice@example.com	2025-03-15	Active
4	charlie77	charlie@example.com	2025-01-18	Active
6	emma55	emma@example.com	2025-04-01	Active
7	frank90	frank@example.com	2025-05-12	Active
9	harry33	harry@example.com	2025-02-28	Active
10	isabella44	isabella@example.com	2025-06-01	Active

```
6 rows in set (0.02 sec)
```

8.3 UPDATE Operation

Requirement

Change the status of user johndoe123 from Inactive to Active.

Query

UPDATE Users

SET status='Active'

WHERE username='johndoe123'

AND status='Inactive';

Output Screenshot

```
mysql> DELETE FROM Users WHERE status = 'Inactive' AND user_id NOT IN ( SELECT DISTINCT user_id FROM WatchLog );
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> SELECT * FROM Users;
```

user_id	username	email	join_date	status
1	johndoe123	john@example.com	2025-02-10	Active
2	alice01	alice@example.com	2025-03-15	Active
3	bob22	bob@example.com	2024-11-20	Active
4	charlie77	charlie@example.com	2025-01-18	Active
5	david89	david@example.com	2024-09-05	Inactive
6	emma55	emma@example.com	2025-04-01	Active
7	frank90	frank@example.com	2025-05-12	Active
8	grace11	grace@example.com	2023-12-25	Inactive
9	harry33	harry@example.com	2025-02-28	Active
10	isabella44	isabella@example.com	2025-06-01	Active

```
10 rows in set (0.00 sec)
```

8.4 DELETE Operation

Requirement

Delete inactive users who have never watched a movie.

Query

```
DELETE FROM Users

WHERE status='Inactive'

AND user_id NOT IN

(

    SELECT DISTINCT user_id

    FROM WatchLog

);
```

Output Screenshot

```
mysql> DELETE FROM Users
-> WHERE status = 'Inactive'
-> AND user_id NOT IN (
->     SELECT DISTINCT user_id
->     FROM WatchLog
-> );
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT *
-> FROM Users
-> WHERE username = 'robert99';
Empty set (0.00 sec)

mysql> Select * from Users;
```

	user_id	username	email	join_date	status
1	1	john doe123	john@example.com	2025-02-10	Active
2	2	alice01	alice@example.com	2025-03-15	Active
3	3	bob22	bob@example.com	2024-11-20	Active
4	4	charlie77	charlie@example.com	2025-01-18	Active
5	5	david89	david@example.com	2024-09-05	Inactive
6	6	emma55	emma@example.com	2025-04-01	Active
7	7	frank90	frank@example.com	2025-05-12	Active
8	8	grace11	grace@example.com	2023-12-25	Inactive
9	9	harry33	harry@example.com	2025-02-28	Active
10	10	isabella44	isabella@example.com	2025-06-01	Active

```
10 rows in set (0.00 sec)
```

9. Analytical JOIN Operation

Genre Popularity Report

Objective

Determine the most popular movie genres based on actual viewing history.

SQL Query

```
SELECT

    m.genre AS Genre_Name,

    COUNT(w.log_id) AS Total_Watch_Count

FROM Movies m

INNER JOIN WatchLog w

ON m.movie_id = w.movie_id

GROUP BY m.genre

ORDER BY Total_Watch_Count DESC;
```


Output Screenshot

```
mysql> SELECT m.genre AS Genre_Name, COUNT(w.log_id) AS Total_Watch_Count FROM Movies m INNER JOIN WatchLog w ON m.movie_id = w.movie_id GROUP BY m.genre ORDER BY Total_Watch_Count DESC;
```

Genre_Name	Total_Watch_Count
Romance	3
Action	2
Drama	2
Comedy	2
Sci-Fi	1

```
5 rows in set (0.01 sec)

mysql>
```

10. Results and Discussion

The implemented database successfully stores movie information, user details, and watch history in a structured format.

Key observations:

- CRUD operations were executed successfully.
- Foreign key relationships ensured data integrity.
- The JOIN query generated useful business insights.
- Popular genres can be identified for future content acquisition.
- Centralized storage reduced redundancy and improved efficiency.

11. Solution Provided

The proposed relational database solution addresses all business challenges faced by CineStream by:

- Centralizing data management
- Eliminating duplicate records
- Improving data consistency
- Supporting efficient reporting
- Enabling analytical decision-making
- Providing scalability for future growth

12. Conclusion

The CineStream Backend Database Project demonstrates the practical implementation of a relational database system using SQL. The designed schema effectively manages movies, users, and watch history while supporting CRUD operations and analytical reporting. The solution improves operational efficiency and provides valuable insights that can help CineStream make informed business decisions.